```
# ======================================================================
# CORE FUNCTIONS FOR DLS
# ======================================================================

def get_neighbors(state):
    """Generates valid adjacent states by moving the blank tile (0) UP, DOWN, LEFT, or RIGHT."""
    neighbors = []
    blank_idx = state.index(0)
    row, col = divmod(blank_idx, 3)

    moves = {
        "UP": (row - 1, col),
        "DOWN": (row + 1, col),
        "LEFT": (row, col - 1),
        "RIGHT": (row, col + 1)
    }

    for move_name, (next_row, next_col) in moves.items():
        if 0 <= next_row < 3 and 0 <= next_col < 3:
            new_blank_idx = next_row * 3 + next_col
            new_state = list(state)
            new_state[blank_idx], new_state[new_blank_idx] = new_state[new_blank_idx], new_state[blank_idx]
            neighbors.append((tuple(new_state), move_name))

    return neighbors

def dls_solve(current_state, goal_state, limit, path, visited):
    """Solves the 8-puzzle using Depth Limited Search (DLS)."""
    # Base case: Goal reached
    if current_state == goal_state:
        return path

    # Base case: Depth bound reached
    if limit <= 0:
        return "CUTOFF"  # Reached limit without finding target

    cutoff_occurred = False

    for neighbor, move in get_neighbors(current_state):
        if neighbor not in visited:
            visited.add(neighbor)

            # Recursively search down with a decremented depth limit boundary
            result = dls_solve(neighbor, goal_state, limit - 1, path + [move], visited)

            visited.remove(neighbor)  # Backtracking step

            if result == "CUTOFF":
                cutoff_occurred = True
            elif result is not None:
                return result

    return "CUTOFF" if cutoff_occurred else None

def take_input(state_name):
    """Takes input row-by-row and strictly validates exactly 3 numbers per row."""
    print(f"\n--- Enter values for the {state_name} (Use 0 for the blank space) ---")
    state = []
    for i in range(3):
        while True:
            try:
                row_input = input(f"Enter row {i+1} values separated by space (e.g., 1 2 3): ").strip().split()
                row_nums = [int(x) for x in row_input]
                if len(row_nums) != 3:
                    print("⚠️ Error: You must enter EXACTLY 3 numbers. Try again.")
                    continue
                state.extend(row_nums)
                break
            except ValueError:
                print("⚠️ Error: Please enter valid numbers only. Try again.")
    return tuple(state)

# ======================================================================
# RUNTIME CONTROL
# ======================================================================
```

```
if __name__ == "__main__":
    initial_layout = take_input("INITIAL STATE")
    goal_layout = take_input("GOAL STATE")

    # User prompts to dictate the exact depth bound constraint
    depth_limit = int(input("\nEnter the Depth Limit (e.g., 5 or 6): "))

    print(f"\nProcessing Depth Limited Search (DLS) with limit={depth_limit}... Please wait...")

    visited_set = set([initial_layout])
    dls_path = dls_solve(initial_layout, goal_layout, depth_limit, [], visited_set)

    print("\n--- SOLUTION MOVES ---")
    if dls_path and dls_path != "CUTOFF":
        print(f"Total steps required: {len(dls_path)}")
        print(" -> ".join(dls_path))
    elif dls_path == "CUTOFF":
        print(f"Cutoff reached! Goal could not be found within depth limit of {depth_limit}.")
    else:
        print("No solution exists for these states.")
```

```
--- Enter values for the INITIAL STATE (Use 0 for the blank space) ---
Enter row 1 values separated by space (e.g., 1 2 3): 2 8 3
Enter row 2 values separated by space (e.g., 1 2 3): 1 6 4
Enter row 3 values separated by space (e.g., 1 2 3): 7 0 5

--- Enter values for the GOAL STATE (Use 0 for the blank space) ---
Enter row 1 values separated by space (e.g., 1 2 3): 1 2 3
Enter row 2 values separated by space (e.g., 1 2 3): 8 0 4
Enter row 3 values separated by space (e.g., 1 2 3): 7 6 5

Enter the Depth Limit (e.g., 5 or 6): 2

Processing Depth Limited Search (DLS) with limit=2... Please wait...

--- SOLUTION MOVES ---
Cutoff reached! Goal could not be found within depth limit of 2.
```

```
# ======================================================================
# CORE FUNCTIONS
# ======================================================================

def get_neighbors(state):
    """Generates valid adjacent states by moving the blank tile (0) UP, DOWN, LEFT, or RIGHT."""
    neighbors = []
    blank_idx = state.index(0)
    row, col = divmod(blank_idx, 3)

    moves = {
        "UP": (row - 1, col),
        "DOWN": (row + 1, col),
        "LEFT": (row, col - 1),
        "RIGHT": (row, col + 1)
    }

    for move_name, (next_row, next_col) in moves.items():
        if 0 <= next_row < 3 and 0 <= next_col < 3:
            new_blank_idx = next_row * 3 + next_col
            new_state = list(state)
            new_state[blank_idx], new_state[new_blank_idx] = new_state[new_blank_idx], new_state[blank_idx]
            neighbors.append((tuple(new_state), move_name))

    return neighbors

def dfs_solve(start_state, goal_state):
    """Solves the 8-puzzle using a memory-safe Graph-DFS to prevent RAM crashes."""
    start_tuple = tuple(start_state)
    goal_tuple = tuple(goal_state)

    stack = [start_tuple]
    visited = set([start_tuple])
    parent_map = {start_tuple: (None, None)}

    while stack:
        current_state = stack.pop()
```

```
        if current_state == goal_tuple:
            path = []
            while current_state != start_tuple:
                curr_parent, move = parent_map[current_state]
                path.append(move)
                current_state = curr_parent
            return path[::-1]

        for neighbor, move in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                parent_map[neighbor] = (current_state, move)
                stack.append(neighbor)

    return None

def take_input(state_name):
    """Takes input row-by-row and strictly validates exactly 3 numbers per row."""
    print(f"\n--- Enter values for the {state_name} (Use 0 for the blank space) ---")
    state = []
    for i in range(3):
        while True:
            try:
                row_input = input(f"Enter row {i+1} values separated by space (e.g., 1 2 3): ").strip().split()
                row_nums = [int(x) for x in row_input]

                if len(row_nums) != 3:
                    print("⚠️ Error: You must enter EXACTLY 3 numbers. Try again.")
                    continue

                state.extend(row_nums)
                break
            except ValueError:
                print("⚠️ Error: Please enter valid numbers only. Try again.")

    return tuple(state)

# ======================================================================
# RUNTIME CONTROL
# ======================================================================

if __name__ == "__main__":
    initial_layout = take_input("INITIAL STATE")
    goal_layout = take_input("GOAL STATE")

    print("\nProcessing Depth First Search (DFS)... Please wait...")
    dfs_path = dfs_solve(initial_layout, goal_layout)

    print("\n--- SOLUTION MOVES ---")
    if dfs_path:
        print(f"Total steps required: {len(dfs_path)}")
        print(" -> ".join(dfs_path))
    else:
        print("No solution found or states are mathematically invalid.")
```

```
--- Enter values for the INITIAL STATE (Use 0 for the blank space) ---
Enter row 1 values separated by space (e.g., 1 2 3): 2 8 3
Enter row 2 values separated by space (e.g., 1 2 3): 1 6 4
Enter row 3 values separated by space (e.g., 1 2 3): 7 0 5

--- Enter values for the GOAL STATE (Use 0 for the blank space) ---
Enter row 1 values separated by space (e.g., 1 2 3): 1 2 3
Enter row 2 values separated by space (e.g., 1 2 3): 8 0 4
Enter row 3 values separated by space (e.g., 1 2 3): 7 6 5

Processing Depth First Search (DFS)... Please wait...

--- SOLUTION MOVES ---
Total steps required: 3437
RIGHT -> UP -> LEFT -> LEFT -> DOWN -> RIGHT -> RIGHT -> UP -> LEFT -> LEFT -> DOWN -> RIGHT -> RIGHT -> UP -> LEFT -> LEFT -> D
```